Programming Fundamentals

Assignment 3 – Part 2 | Solutions

Instructor: Umar Khalil

Problem 1: 2nd Highest and 2nd Lowest Values in an Array

Approach

We iterate through the array once to find the lowest and second lowest, and again for highest and second highest. We track distinct values to avoid treating duplicates as separate entries.

Code

```
#include <iostream>
using namespace std;

int calculate2Low(int arr[], int n) {
    int lowest = INT_MAX, secondLowest = INT_MAX;
    for (int i = 0; i < n; i++) {
        if (arr[i] < lowest) {
            secondLowest = lowest;
            lowest = arr[i];
        } else if (arr[i] < secondLowest && arr[i] != lowest) {
            secondLowest = arr[i];
        }
    }
    return secondLowest;
}

int calculate2High(int arr[], int n) {
    int highest = INT_MIN, secondHighest = INT_MIN;
    for (int i = 0; i < n; i++) {
        if (arr[i] > highest) {
            secondHighest = highest;
            highest = arr[i];
        } else if (arr[i] > secondHighest && arr[i] != highest) {
            secondHighest = arr[i];
        }
    }
    return secondHighest;
}

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    int scores[n];
    cout << "Enter scores: ";
    for (int i = 0; i < n; i++) cin >> scores[i];

    cout << "2nd Lowest: "  << calculate2Low(scores, n)  << endl;
    cout << "2nd Highest: " << calculate2High(scores, n) << endl;
    return 0;
```

```
}
```

Sample Output

```
Enter number of students: 5
Enter scores: 45 78 23 67 89
2nd Lowest: 45
2nd Highest: 78
```

Problem 2: Sum of an Array

Approach

A simple loop accumulates the sum into a double variable, and we print it formatted to 2 decimal places using fixed and precision(2).

Code

```
#include <iostream>
using namespace std;

double calculateSum(int arr[], int n) {
    double sum = 0;
    for (int i = 0; i < n; i++) sum += arr[i];
    return sum;
}

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    int scores[n];
    cout << "Enter scores: ";
    for (int i = 0; i < n; i++) cin >> scores[i];

    cout << fixed;
    cout.precision(2);
    cout << "Sum: " << calculateSum(scores, n) << endl;
    return 0;
}
```

Sample Output

```
Enter number of students: 4
Enter scores: 80 90 70 60
Sum: 300.00
```

Problem 3: Temperature Converter

Approach

The formula (C × 9/5) + 32 converts Celsius to Fahrenheit. Using 9.0/5.0 ensures floating-point division. Output is rounded to 1 decimal place.

Code

```
#include <iostream>
using namespace std;

double celsiusToFahrenheit(double c) {
    return (c * 9.0 / 5.0) + 32;
}

int main() {
    double celsius;
    cout << "Enter temperature in Celsius: ";
    cin >> celsius;

    cout << fixed;
    cout.precision(1);
    cout << "Fahrenheit: " << celsiusToFahrenheit(celsius) << endl;
    return 0;
}
```

Sample Output

```
Enter temperature in Celsius: 100
Fahrenheit: 212.0
```

Problem 4: Complete Program – Leap Year

Approach

A year is a leap year if it is divisible by 400, OR divisible by 4 but not by 100. Both conditions are combined into one if statement using the logical OR (||) and AND (&&) operators.

Completed Code

```
#include <iostream>
using namespace std;

bool isLeapYear(int year) {
    if (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)) {
        return true;
    }
    return false;
}

int main() {
    cout << isLeapYear(2024);
    return 0;
}
```

Filled-in Condition

```
year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)
```

Sample Output

```
1
```

Problem 5: Vowel Counter

Approach

We loop through the character array until the null terminator '\0' is reached. Inside the loop, we check if the current character is a vowel (both lowercase and uppercase) using the || operator.

Code

```
#include <iostream>
using namespace std;

int countVowels(char word[]) {
    int count = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        char c = word[i];
        if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
            c=='A' || c=='E' || c=='I' || c=='O' || c=='U') {
            count++;
        }
    }
    return count;
}

int main() {
    char name[100];
    cout << "Enter a word: ";
    cin >> name;
    cout << "Total Vowels: " << countVowels(name) << endl;
    return 0;
}
```

Sample Output

```
Enter a word: Shahzaman
Total Vowels: 3
```

Problem 6: Alphabetical Sorter

Approach

Bubble sort compares adjacent characters using arr[j] > arr[j+1]. If they are out of order, they are swapped using a temporary variable. Two nested loops handle all passes.

Code

```
#include <iostream>
using namespace std;
```

```
int main() {
    char arr[5];
    cout << "Enter 5 characters: ";
    for (int i = 0; i < 5; i++) cin >> arr[i];

    // Bubble Sort
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                char temp = arr[j];
                arr[j]     = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    cout << "Sorted Letters: ";
    for (int i = 0; i < 5; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

Sample Output

```
Enter 5 characters: e a c b d
Sorted Letters: a b c d e
```